

Привет. Как открыть портативный git через cmd?

Чтобы открыть и использовать портативный Git (Portable Git) через стандартную командную строку (cmd), вам нужно **запустить файл `git-cmd.exe` из папки программы или временно прописать путь к исполняемому файлу в переменную окружения PATH.**  Reddit · r/github

Ниже приведены три простых способа сделать это, в зависимости от ваших задач.

Способ 1. Быстрый запуск встроенной консоли Git

Внутри папки с портативным Git уже есть настроенная командная строка, которая сразу видит все команды git.

1. Откройте **cmd**.
2. Перейдите в папку с портативным Git с помощью команды `cd` :

cmd

```
cd /d "C:\Путь_К_Папке\PortableGit"
```

Используйте код с осторожностью.



3. Запустите специальный файл оболочки:

cmd

```
git-cmd.exe
```

Используйте код с осторожностью.



 Reddit · r/github

После этого откроется новое окно командной строки, где команда `git` будет работать в любой папке.

Способ 2. Использование команд Git в текущем окне cmd

Если вы хотите работать в уже открытом окне cmd и не хотите запускать новые окна, добавьте путь к Git в переменную окружения текущей сессии:

1. Введите в cmd команду (замените путь на свой, важно указать именно подпапку `\cmd`):

cmd

```
set PATH="C:\Путь_К_Папке\PortableGit\cmd";%PATH%
```

Используйте код с осторожностью.



2. Проверьте работу программы:

cmd

```
git --version
```

Используйте код с осторожностью.



Этот способ работает только внутри того окна cmd, где вы ввели команду `set`. После закрытия консоли настройка сбросится.

Способ 3. Прямой вызов без настройки путей

Вы можете вызвать Git из любой папки, просто указав полный путь к исполняемому файлу в кавычках:  [Reddit · r/learnpython](#)

cmd

```
"C:\Путь_К_Папке\PortableGit\cmd\git.exe" status
```

Используйте код с осторожностью.



Если вы хотите, чтобы портативный Git работал в cmd **всегда** (даже после перезагрузки компьютера) без ввода лишних команд, его нужно добавить в системные переменные PATH. Рассказать, как настроить такую **постоянную интеграцию** в Windows?



“ “C:\Путь_К_Папке\PortableGit\cmd\git.exe" status

расскажи подробнее

Этот способ позволяет выполнять любые команды Git в любой папке вашего компьютера без предварительной настройки путей. Вы просто заменяете стандартное слово `git` на полный путь к файлу `git.exe`.

Как это работает на практике

1. Откройте **cmd** и перейдите в папку с вашим проектом (где лежит репозиторий):

cmd

```
cd /d "C:\Users\User\Projects\MyApp"
```

Используйте код с осторожностью.



2. Вместо привычной команды `git status` введите путь к портативной версии в кавычках, а затем саму команду:

cmd

```
"C:\PortableGit\cmd\git.exe" status
```

Используйте код с осторожностью.



3. Точно так же выполняются и все остальные команды:

cmd

```
"C:\PortableGit\cmd\git.exe" add .  
"C:\PortableGit\cmd\git.exe" commit -m "First commit"  
"C:\PortableGit\cmd\git.exe" push origin main
```



Важные технические нюансы

- **Почему подпапка `\cmd`, а не `\bin`?** Внутри PortableGit есть две папки с файлом `git.exe`. Нужно использовать именно `PortableGit\cmd\git.exe`. Если запустить файл из папки `\bin`, встроенные утилиты (например, SSH для связи с GitHub) могут работать некорректно.
- **Зачем нужны кавычки?** Кавычки обязательны, если в пути к вашей папке есть пробелы (например, `"C:\Program Files\PortableGit\cmd\git.exe"`). Если пробелов нет, кавычки можно опустить, но лучше их ставить всегда во избежание ошибок cmd.

Как упростить ввод (Создание псевдонима)

Чтобы не писать длинный путь каждый раз, вы можете создать временный псевдоним (alias) прямо в текущей сессии cmd с помощью команды `doskey`:

cmd

```
doskey git="C:\PortableGit\cmd\git.exe" %*
```

Используйте код с осторожностью.



После этого в данном окне cmd вы сможете писать просто `git status`, `git log` и так далее. Опция `%*` указывает командной строке автоматически подставлять любые аргументы, которые вы вводите после слова `git`.

Хотите узнать, как автоматизировать этот процесс, чтобы **создать ярлык** для автоматического запуска cmd с уже готовым доступом к Git?



Команда `source` — это встроенный инструмент командной оболочки Bash (которая используется в Git Bash). Она служит для **выполнения скриптов в контексте текущего окна консоли** [1, 2].

Вместо того чтобы запускать скрипт как отдельную изолированную программу, `source` считывает команды из файла и выполняет их так, будто вы сами ввели их по очереди на клавиатуре [2, 3].

Зачем она нужна в Git Bash?

Чаще всего в Git Bash команду `source` применяют в двух ситуациях:

1. Применение настроек без перезапуска консоли.

Если вы отредактировали конфигурационный файл профиля (например, `.bashrc` или `.bash_profile`, где хранятся ваши личные псевдонимы (aliases) и переменные), изменения не вступят в силу автоматически. Чтобы не закрывать и не открывать Git Bash заново, вы пишете:

```
bash
```

```
source ~/.bashrc
```

Используйте код с осторожностью.



2. Активация виртуальных окружений (например, в Python).

Когда вы создаете изолированное окружение для проекта, его активация происходит именно через `source`, чтобы переменные пути применились к текущей сессии:

```
bash
```

```
source venv/Scripts/activate
```

Используйте код с осторожностью.



Синоним команды

В экосистеме Linux и Bash у команды `source` есть абсолютно идентичный, но более короткий синоним — **точка (.)** [1]. Две эти команды делают абсолютно одно и то же:

- `source ~/.bashrc`
- `. ~/.bashrc` (между точкой и путем обязательно должен быть пробел) [1].

В чем ключевое отличие от обычного запуска скрипта?

- **Обычный запуск** (например, `./myscript.sh`): Git Bash создает «дочерний» процесс (новую скрытую вкладку) [2, 3]. Все переменные, созданные внутри скрипта, стираются, как только он завершает работу [2]. Текущее окно консоли о них ничего не узнает [2].
- **Запуск через `source myscript.sh`**: Скрипт выполняется прямо в текущем окне [2, 3]. Если внутри скрипта была создана переменная или изменен путь к папке, эти **изменения останутся активными** в вашей консоли после завершения работы скрипта [2].

Хотите, покажу пример, как с помощью `source` создать **файл с собственными быстрыми командами (alias)** для Git, чтобы экономить время при вводе?



Есть ли аналог `source` в windows cmd?

Да, в стандартной командной строке Windows (cmd) аналогом `source` является команда `call`.

Она выполняет команды из другого пакетного файла (`.bat` или `.cmd`) внутри **текущего контекста**. Все переменные окружения и настройки, которые изменятся внутри вызванного скрипта, сохранятся в вашем текущем окне cmd [microsoft.com].

Разница в поведении (`call` против обычного запуска)

Представьте, что у вас есть скрипт `setenv.bat`, который создает переменную: `set MY_VAR=Hello`.

- **Обычный запуск (без `call`)**:
Если вы просто введете `setenv.bat`, то cmd передаст управление этому файлу. Скрипт выполнится, но текущий сеанс командной строки

сразу завершит работу или «забудет» переменные. Вы не сможете использовать `%MY_VAR%` дальше.

- Запуск через `call` :

cmd

```
call setenv.bat
```

Используйте код с осторожностью.



Управление передается файлу `setenv.bat` , он отработывает, а затем `cmd` возвращает вас в текущую строку. Переменная `%MY_VAR%` остается доступной для дальнейшей работы.

Живой пример: Активация окружения Python

В Git Bash вы используете `source` :

bash

```
source venv/Scripts/activate
```

Используйте код с осторожностью.



В Windows `cmd` вы используете `call` (хотя в случае с Python-окружением в `cmd` скрипт написан так, что его можно вызвать и напрямую, `call` гарантирует правильный возврат в сессию) [python.org]:

cmd

```
call venv\Scripts\activate.bat
```

Используйте код с осторожностью.



Альтернатива для PowerShell

Если вы когда-нибудь перейдете с `cmd` на **PowerShell**, то там аналогом `source` является **Dot Sourcing** (запуск через точку) [microsoft.com]:

powershell

```
.. \script.ps1
```

Используйте код с осторожностью.



Хотите посмотреть, как написать простой `.bat` скрипт для cmd, который будет автоматически **подключать ваш портативный Git** при запуске, используя механику `call` ?

